

PLANO DE QUALIDADE

E-Commerce de Camisetas

Plano de Testes e Requisitos Não-Funcionais

Stack	Java 21 + Spring Boot 3.x + Apache Kafka + Docker
Curso	Análise e Desenvolvimento de Sistemas — 4º Período
Semestre	2026/1

Sumário

1.	Introdução e Objetivos de Qualidade	3
2.	Requisitos Não-Funcionais (RNF)	4
3.	Plano de Testes — Visão Geral	6
4.	Cenários de Teste — Unitários	7
5.	Cenários de Teste — Integração	9
6.	Cenários de Teste — End-to-End	10
7.	Cenários de Teste — Não-Funcionais	11
8.	Ferramentas e Configuração	12
9.	Critérios de Aceite e Cobertura	13

1. Introdução e Objetivos de Qualidade

Este documento define o Plano de Testes e os Requisitos Não-Funcionais do e-commerce de camisetas desenvolvido como Projeto Integrador do curso de ADS. O sistema é composto por microsserviços Spring Boot comunicando-se de forma assíncrona via Apache Kafka, com frontend React recebendo atualizações em tempo real via WebSocket.

Objetivo principal: garantir que o fluxo assíncrono Pedido → Kafka → Notificação funcione de forma confiável, desacoplada e com tempo de resposta satisfatório, atendendo os critérios de avaliação da N2 (0,4 pts mensageria + 0,4 pts testes).

1.1 Escopo dos Testes

Serviço	O que será testado
product-service	CRUD de produtos, validações de entrada, persistência
logistics-service	Criação de pedidos, publicação de eventos Kafka, máquina de estados
notification-service	Consumo de eventos Kafka, entrega via WebSocket
admin-service	Listagem de pedidos, métricas do dashboard
Infra / Mensageria	Kafka Producer/Consumer, reconexão, tolerância a falhas

1.2 Tipos de Teste Adotados

Tipo	Descrição
Unitário	Testa cada classe isolada com mocks. Rápido, sem dependências externas.
Integração	Testa a comunicação entre camadas com banco e Kafka reais via Testcontainers.
End-to-End	Valida o fluxo completo do sistema, do frontend ao banco.
Não-Funcional	Valida desempenho, segurança, disponibilidade e manutenibilidade.

2. Requisitos Não-Funcionais (RNF)

Os Requisitos Não-Funcionais definem as características de qualidade do sistema. Cada RNF possui critério de aceite mensurável para validação objetiva na banca.

2.1 Desempenho

ID	Nome	Critério de Aceite	Como Validar
RNF-01	Tempo de resposta API	Endpoints REST respondem em até 500ms para 95% das requisições em carga normal.	Teste de carga com 50 usuários simultâneos por 60 segundos.
RNF-02	Latência do fluxo Kafka	O evento publicado no Kafka deve ser consumido e entregue via WebSocket em até 2 segundos.	Medir timestamp de publicação vs. recebimento no frontend.
RNF-03	Throughput mínimo	O sistema deve processar no mínimo 20 pedidos por segundo sem degradação.	Teste de carga com Apache JMeter ou k6.

2.2 Disponibilidade e Confiabilidade

ID	Nome	Critério de Aceite	Como Validar
RNF-04	Disponibilidade	O sistema deve estar disponível 99% do tempo durante o período de avaliação.	Monitorar com Spring Actuator /health durante as demos.
RNF-05	Tolerância a falhas Kafka	Se o notification-service reiniciar, ele deve recuperar os eventos perdidos ao voltar.	Derrubar o container, publicar eventos, subir novamente e verificar consumo.
RNF-06	Persistência de dados	Pedidos criados devem persistir mesmo após restart dos serviços.	Criar pedido, reiniciar o container order-db, consultar o pedido.

2.3 Segurança

ID	Nome	Critério de Aceite	Como Validar
RNF-07	Proteção de credenciais	Nenhuma senha, chave ou segredo deve estar hardcoded no código-fonte ou subir ao GitHub.	Revisar todos os commits. Credenciais apenas no .env (não versionado).
RNF-08	CORS configurado	O backend deve rejeitar requisições de origens não autorizadas.	Fazer requisição de origem diferente e verificar resposta 403.
RNF-09	Validação de entrada	Dados inválidos enviados à API devem retornar HTTP 400 com mensagem descritiva.	Enviar payload sem campos obrigatórios e verificar resposta.

2.4 Manutenibilidade

ID	Nome	Critério de Aceite	Como Validar
RNF-10	Cobertura de testes	Cobertura mínima de 70% nas classes da camada service de cada microserviço.	Relatório gerado pelo JaCoCo via Maven: mvn verify.
RNF-11	Separação de responsabilidades	O pacote domain/ não deve importar nenhuma classe do Spring Framework.	Revisar imports. Zero dependências de framework no domínio.
RNF-12	Configuração externalizada	Toda configuração de ambiente deve estar em variáveis de ambiente (.env), nunca no código.	Trocar valor do .env e verificar que o sistema adota sem recompilar.

2.5 Portabilidade

ID	Nome	Critério de Aceite	Como Validar
RNF-13	Containerização	Todo o ambiente deve subir com um único comando: docker compose up -d.	Clonar o repositório em máquina limpa e executar o comando.
RNF-14	Independência de SO	O projeto deve funcionar em Windows, macOS e Linux sem alterações no código.	Testar em ao menos dois sistemas operacionais distintos.

3. Plano de Testes — Visão Geral

A estratégia de testes segue a pirâmide de testes: maior quantidade de testes unitários (rápidos e baratos), quantidade menor de testes de integração, e poucos testes end-to-end (lentos mas de alto valor para a banca).

Tipo	Ferramentas	O que cobre	Qtd. est.	Duração	Quando rodar
Unitário	JUnit 5 + Mockito	Service, Factory, Domain	~50	< 10s	A cada commit
Integração	Spring Boot Test + Testcontainers	Repository, Kafka, REST	~20	< 60s	A cada PR
End-to-End	REST Assured + Testcontainers	Fluxo completo	~10	< 3min	Antes da entrega
Não-Funcional	JMeter / k6	Carga e latência	~5	~5min	Antes da banca

Para a N2: o que o professor vai querer ver é o relatório do JaCoCo mostrando cobertura $\geq 70\%$ na camada service, e pelo menos um teste de integração com Testcontainers provando que o Kafka funciona de ponta a ponta.

4. Cenários de Teste — Unitários

Testam uma única classe em isolamento. Todas as dependências externas (banco, Kafka, outros serviços) são substituídas por mocks com Mockito.

4.1 product-service

ID	Cenário	Entrada	Resultado Esperado	Status
CT-U-01	Listar produtos ativos	Repositório retorna lista com 3 produtos ativos	Service retorna lista com 3 ProductResponse	PASS
CT-U-02	Buscar produto por ID inexistente	Repositório retorna Optional.empty()	Service lança ResourceNotFoundException com HTTP 404	PASS
CT-U-03	Criar produto com dados válidos	Request com nome, preço e estoque preenchidos	Repositório chamado 1x; retorna ProductResponse com ID	PASS
CT-U-04	Criar produto com preço negativo	Request com price = -10.00	Lança ConstraintViolationException antes de chamar repositório	PASS
CT-U-05	Deletar produto — soft delete	Produto existe no repositório	Campo active = false; repositório.save() chamado 1x	PASS

4.2 logistics-service (order-service)

ID	Cenário	Entrada	Resultado Esperado	Status
CT-U-06	Criar pedido com itens válidos	OrderRequest com 2 itens e email válido	Order salvo com status Processing; KafkaTemplate.send() chamado 1x	PASS
CT-U-07	Criar pedido sem itens	OrderRequest com lista de itens vazia	Lança IllegalArgumentException; repositório NÃO é chamado	PASS
CT-U-08	Atualizar status para Shipped	Pedido existente com status Confirmed	Status atualizado para Shipped; evento order.updated publicado	PASS
CT-U-09	OrderFactory — construção do agregado	OrderRequest com 3 itens	Order criado com totalPrice = soma dos itens; status = Processing	PASS
CT-U-10	Sealed interface — pattern matching	OrderStatus = Delivered	switch retorna label 'Entregue!' sem lançar exceção	PASS
CT-U-11	Calcular total do pedido	2 itens: R\$49,90 x2 e R\$79,90 x1	totalPrice = R\$179,70	PASS

4.3 notification-service

ID	Cenário	Entrada	Resultado Esperado	Status
CT-U-12	Consumir evento e enviar WebSocket	OrderEvent com orderId e status 'Shipped'	messagingTemplate.convertAndSend() chamado com /topic/orders	PASS
CT-U-13	Consumir evento com payload nulo	Mensagem Kafka com body null	Lança exceção controlada; não trava o consumer	PASS

Exemplo de código — CT-U-06:

```
when(orderRepository.save(any())).thenReturn(savedOrder);  
when(productClient.findById(any())).thenReturn(productResponse);  
orderService.create(request);
```

```
verify(kafkaTemplate, times(1)).send(eq("order.created"), any());
```


5. Cenários de Teste — Integração

Usam Testcontainers para subir PostgreSQL e Kafka reais em Docker durante a execução dos testes. Validam a comunicação real entre as camadas do sistema.

Por que Testcontainers? Garante que o teste roda com as mesmas versões do banco e do Kafka que rodam em produção. Elimina o 'funciona na minha máquina'.

ID	Cenário	Entrada	Resultado Esperado	Infra
CT-I-01	Criar pedido e verificar persistência	POST /api/orders com payload válido	HTTP 201; pedido encontrado no PostgreSQL; status = PROCESSING	Testcontainers PG
CT-I-02	Publicar evento no Kafka e consumir	OrderServiceImpl.create() chamado	Evento encontrado no tópico order.created em até 5s	Testcontainers Kafka
CT-I-03	Fluxo Kafka completo no teste	Producer publica no tópico hello.world	Consumer recebe a mensagem; contador de mensagens = 1	Testcontainers Kafka
CT-I-04	Buscar produto pelo endpoint REST	GET /api/products/{id} com produto no banco	HTTP 200 com JSON contendo todos os campos do produto	Testcontainers PG
CT-I-05	Validação de entrada — endpoint REST	POST /api/orders com body vazio {}	HTTP 400 com lista de erros de validação	Testcontainers PG
CT-I-06	Reconexão do consumer após restart	Kafka reinicia; producer envia mensagem; consumer volta	Consumer recebe a mensagem pendente (auto-offset-reset=earliest)	Testcontainers Kafka

5.1 Configuração Testcontainers

```
@SpringBootTest
@Testcontainers
class OrderIntegrationTest {

    @Container
    static PostgreSQLContainer postgres =
        new PostgreSQLContainer<>("postgres:16");

    @Container
    static KafkaContainer kafka =
        new KafkaContainer(
            DockerImageName.parse("confluentinc/cp-kafka:7.5.0"));
}
```

6. Cenários de Teste — End-to-End

Validam o sistema completo em execução. Simulam o comportamento real do usuário e provam que todos os serviços funcionam juntos. São os testes mais importantes para a demonstração na banca.

ID	Cenário	Entrada	Resultado Esperado	Prioridade
CT-E-01	Fluxo completo de pedido	Cliente cria pedido via POST /api/orders	Pedido salvo no banco, evento no Kafka, WebSocket envia update ao frontend	Alta
CT-E-02	Atualização de status em tempo real	Admin atualiza status para 'Shipped' via PATCH /api/admin/orders/{id}/status	Frontend recebe atualização via WebSocket sem recarregar a página	Alta
CT-E-03	Hello World arquitetural	Clique no botão do frontend que chama POST /api/hello	Mensagem percorre Frontend → Logistics → Kafka → Notification → WebSocket	Alta
CT-E-04	Listagem de produtos na vitrine	GET /api/products retornado pelo Gateway na porta 8080	HTTP 200 com lista de produtos; Gateway roteou corretamente	Média
CT-E-05	Resiliência — notification-service reinicia	Derrubar notification-service, criar pedido, subir novamente	Pedido criado com sucesso; ao subir, consumer processa evento pendente	Média

Para a banca (CT-E-03): abrir o browser em localhost:5173, mostrar o badge 'WebSocket conectado', clicar no botão e mostrar a mensagem aparecendo em tempo real. Esse cenário sozinho prova toda a arquitetura funcionando.

7. Cenários de Teste — Não-Funcionais

Validam os Requisitos Não-Funcionais definidos na seção 2. Esses testes garantem que o sistema não apenas funciona corretamente, mas funciona bem.

7.1 Desempenho (RNF-01, RNF-02, RNF-03)

ID	Cenário	Parâmetros	Critério de Aceite	Ferramenta
CT-NF-01	Latência REST em carga	50 usuários, 60s, GET /api/products	P95 < 500ms	k6 ou JMeter
CT-NF-02	Latência ponta a ponta Kafka	Publicar 10 eventos consecutivos	Todos consumidos em < 2s	Log de timestamp
CT-NF-03	Throughput de pedidos	20 POST /api/orders por segundo	Zero erros HTTP 5xx	k6

7.2 Segurança (RNF-07, RNF-08, RNF-09)

ID	Cenário	Como executar	Critério de Aceite	Ferramenta
CT-NF-04	Credenciais no repositório	git log --all grep -i password	Zero ocorrências de senha no histórico Git	Manual
CT-NF-05	CORS — origem não autorizada	Requisição de http://malicious.com	HTTP 403 Forbidden	cURL / Postman
CT-NF-06	Validação de campo obrigatório	POST /api/orders sem campo customerEmail	HTTP 400 com mensagem 'email é obrigatório'	REST Assured

7.3 Portabilidade (RNF-13, RNF-14)

ID	Cenário	Como executar	Critério de Aceite	Ferramenta
CT-NF-07	Subir ambiente do zero	Máquina limpa: git clone + docker compose up -d	Todos containers 'running' em < 5 min	Docker
CT-NF-08	Independência de sistema operacional	Executar em Windows e macOS	Sistema funciona sem alterações	Manual

8. Ferramentas e Configuração

8.1 Stack de Testes

Ferramenta	Finalidade	Como incluir
JUnit 5	Framework principal de testes	Incluso no spring-boot-starter-test
Mockito	Criação de mocks para testes unitários	Incluso no spring-boot-starter-test
Testcontainers	PostgreSQL e Kafka reais nos testes	org.testcontainers:postgresql + kafka
Spring Boot Test	Context de teste com injeção de dependência	Incluso no spring-boot-starter-test
REST Assured	Testes de API REST de forma fluente	io.rest-assured:rest-assured
JaCoCo	Relatório de cobertura de código	Plugin Maven: jacoco-maven-plugin
k6	Testes de carga e desempenho	Instalação separada: k6.io

8.2 Dependências Maven (pom.xml de cada serviço)

```
org.springframework.boot
spring-boot-starter-test
test

org.testcontainers
postgresql
test

org.testcontainers
kafka
test
```

8.3 Configuração JaCoCo — Cobertura mínima 70%

```
org.jacoco
jacoco-maven-plugin

PACKAGE

LINE
COVEREDRATIO
0.70
```

Gerar relatório: ./mvnw verify → relatório em target/site/jacoco/index.html

9. Critérios de Aceite e Cobertura

Define o que precisa estar verde para o projeto ser considerado aprovado em qualidade, alinhado com os critérios de avaliação da N2.

9.1 Critérios mínimos para aprovação

Critério	Como verificar	Tipo
Testes unitários passando	100% dos testes CT-U-01 a CT-U-13 com status PASS	Obrigatório
Cobertura service layer $\geq 70\%$	Relatório JaCoCo com cobertura de linha $\geq 70\%$ no pacote service/	Obrigatório
Teste integração Kafka	CT-I-02 ou CT-I-03 provando consumo real de mensagem	Obrigatório
Fluxo E2E funcionando	CT-E-03 demonstrado ao vivo na banca	Obrigatório
RNFs documentados	Todos os 14 RNFs com critério de aceite definido neste documento	Obrigatório
Latência Kafka $< 2s$	CT-NF-02 executado e registrado em evidência (print de log)	Desejável
Zero credenciais no Git	CT-NF-04 executado e sem ocorrências	Obrigatório

9.2 Rastreabilidade — RNF × Critério de Avaliação

RNFs	Categoria	Critério de Avaliação	Impacto
RNF-01, 02, 03	Desempenho	Inovação e UI Final (0,3 N2)	Suporte
RNF-04, 05, 06	Confiabilidade	Mensageria desacoplada (0,4 N2)	Direto
RNF-07, 08, 09	Segurança	Qualidade de código (0,5 N2)	Direto
RNF-10, 11	Cobertura / Arq. Limpa	Qualidade e Testes (0,4 N2)	Direto
RNF-12	Config. externalizada	Organização (0,2 N1)	Direto
RNF-13, 14	Portabilidade	Docker da Fila/Broker (0,2 N1)	Direto

Resumo executivo: com os 13 cenários unitários, 6 de integração e 3 end-to-end passando, a cobertura JaCoCo acima de 70% e o CT-E-03 demonstrado ao vivo, o projeto atende todos os critérios de qualidade da N2 e está preparado para a arguição dos professores.